

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №6
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Унарные и бинарные операции»

Выполнил:

студенты группы 24ВВВ2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Название

Унарные и бинарные операциями.

Цель работы

Научиться работать с унарными и бинарными операциями.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M1, M2 смежности неориентированных помеченных графов G1, G2. Выведите сгенерированные матрицы на экран.
2. * Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Задание 2

1. Для матричной формы представления графов выполните операцию:
 - а) отождествления вершин
 - б) стягивания ребра
 - в) расщепления вершины
 Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения операции выведите на экран.
2. * Для представления графов в виде списков смежности выполните операцию:
 - а) отождествления вершин
 - б) стягивания ребра
 - в) расщепления вершины
 Номера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения операции выведите на экран.

Задание 3

1. Для матричной формы представления графов выполните операцию:
 - а) объединения $G = G1 \cup G2$
 - б) пересечения $G = G1 \cap G2$
 - в) кольцевой суммы $G = G1 \oplus G2$
 Результат выполнения операции выведите на экран.

Задание 4 *

1. Для матричной формы представления графов выполните операцию декартова произведения графов $G = G1 \times G2$.
Результат выполнения операции выведите на экран.

Описание метода решения задачи

Задание 1

Метод решения задачи основан на матричном представлении графов и их преобразовании в списки смежности. Для каждого графа строится квадратная матрица смежности размером $n \times n$, где n - количество вершин. Элемент матрицы a_{ij} принимает значение 1 при наличии ребра между вершинами i и j , и 0 при его отсутствии. Особенностью метода является обеспечение симметричности

матрицы для неориентированных графов и нулевой главной диагонали. Преобразование в списки смежности выполняется путем построчного анализа матрицы: для каждой вершины i формируется перечень всех вершин j , для которых $a_{ij} = 1$ при $i \neq j$. Метод гарантирует однозначное соответствие между матричным и списковым представлениями графа, обеспечивая полное сохранение информации о структуре связей.

Задание 2

1. Метод решения задачи основан на матричных операциях над графами. Для операции отождествления вершин используется алгоритм объединения двух вершин в одну с сохранением всех инцидентных ребер: создается новая матрица размером $(n-1) \times (n-1)$, где связи объединяемых вершин комбинируются через логическую операцию ИЛИ, а индексы остальных вершин корректируются. Для операции стягивания ребра применяется модифицированное отождествление вершин с предварительной проверкой существования ребра и последующим удалением возможной петли. Операция расщепления вершины реализуется через расширение матрицы до размера $(n+1) \times (n+1)$ со случайным распределением инцидентных ребер между исходной и новой вершиной и обязательным соединением этих вершин. Все операции сохраняют свойство неориентированности графа через симметричное заполнение матрицы.

2. Метод решения задачи основан на операциях со списками смежности для представления графов. Для операции отождествления вершин используется алгоритм слияния двух вершин: создается новый граф с уменьшенным количеством вершин, формируется объединенный список смежности путем сбора всех уникальных соседей обеих вершин с исключением петель, а в остальных списках производится замена ссылок на удаляемую вершину. Операция стягивания ребра реализуется как модификация отождествления с предварительной проверкой существования ребра и последующим удалением возможной петли. Операция расщепления вершины выполняется через создание нового графа с увеличенным количеством вершин со случайным распределением инцидентных ребер между исходной и новой вершиной и обязательным добавлением соединяющего их ребра. Все операции обеспечивают сохранение неориентированности графа через симметричное добавление ребер и исключение дубликатов в списках смежности.

Задание 3

Метод решения задачи основан на поточечных булевых операциях над матрицами смежности графов. Для операции объединения графов применяется логическое ИЛИ к соответствующим элементам матриц: результирующая матрица содержит 1, если ребро присутствует хотя бы в одном из исходных графов. Для операции пересечения используется логическое И: результат содержит 1 только при одновременном наличии ребра в обоих графах. Операция кольцевой суммы реализуется через сложение по модулю 2 (исключающее ИЛИ), где ребро сохраняется только если оно присутствует ровно в одном из графов. Все операции сохраняют свойство симметричности матриц.

Задание 4

Метод решения задачи основан на построении декартова произведения графов через матричное представление. Вершины результирующего графа

образуются как все возможные пары вершин из исходных графов G1 и G2, что дает матрицу размером $|V1| \times |V2|$. Смежность вершин определяется по правилу: две вершины $(u1, v1)$ и $(u2, v2)$ соединяются ребром, если выполняется одно из условий - $u1 = u2$ и существует ребро $(v1, v2)$ в G2 (горизонтальные связи), либо $v1 = v2$ и существует ребро $(u1, u2)$ в G1 (вертикальные связи). Алгоритм последовательно перебирает все четверки вершин $(u1, v1, u2, v2)$, вычисляет соответствующие индексы в результирующей матрице по формуле $i = u1 * |v2| + v1$, и устанавливает связь при выполнении условий смежности. Метод сохраняет свойство неориентированности графов через симметричное заполнение матрицы.

Листинг

Файл 6LR_1.cpp (1 задание)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
//Функция для генерации матрицы смежности
void generateAdjacencyMatrix(int** matrix, int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = i + 1; j < vertices; j++) {
            //Случайное значение 0 или 1
            int value = rand() % 2;
            matrix[i][j] = value;
            matrix[j][i] = value; // Граф неориентированный
        }
        matrix[i][i] = 0;
    }
}

//Функция для вывода матрицы смежности
void printMatrix(int** matrix, int vertices) {
    printf("Матрица смежности:\n");
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

//Функция для преобразования матрицы смежности в список смежности
void matrixToList(int** matrix, int vertices) {
    printf("Список смежности:\n");
    //Строки матрицы
    for (int i = 0; i < vertices; i++) {
        printf("Вершина %d: ", i + 1);
        //Флаг для отслеживания наличия соседей
        int hasNeighbors = 0;
        //Столбцы матрицы
        for (int j = 0; j < vertices; j++) {
            if (matrix[i][j] == 1 && i != j) {
```

```

        printf("%d ", j + 1);
        //Устанавливаем флаг, что у вершины есть хотя бы
один сосед
        hasNeighbors = 1;
    }
}
if (!hasNeighbors) {
    printf("Нет соседей");
}
printf("\n");
}
printf("\n");
}

//Функция для освобождения памяти
void freeMatrix(int** matrix, int vertices) {
    for (int i = 0; i < vertices; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(NULL)); // Инициализация генератора случайных чисел
    int vertices1, vertices2;

    //Ввод количества вершин для первого графа
    printf("Введите количество вершин для графа G1: ");
    scanf("%d", &vertices1);

    //Ввод количества вершин для второго графа
    printf("Введите количество вершин для графа G2: ");
    scanf("%d", &vertices2);

    //Проверка корректности ввода
    if (vertices1 <= 0 || vertices2 <= 0) {
        printf("Ошибка: количество вершин должно быть положительным
числом.\n");
        return 1;
    }

    //Выделение памяти для матрицы M1
    int** M1 = (int**)malloc(vertices1 * sizeof(int*));
    for (int i = 0; i < vertices1; i++) {
        M1[i] = (int*)malloc(vertices1 * sizeof(int));
    }

    //Выделение памяти для матрицы M2
    int** M2 = (int**)malloc(vertices2 * sizeof(int*));
    for (int i = 0; i < vertices2; i++) {
        M2[i] = (int*)malloc(vertices2 * sizeof(int));
    }

    //Генерация и вывод матрицы M1
    printf("\nГраф G1\n");

```

```

generateAdjacencyMatrix(M1, vertices1);
printMatrix(M1, vertices1);

//Преобразование и вывод списка смежности для G1
matrixToList(M1, vertices1);

//Генерация и вывод матрицы M2
printf("Граф G2\n");
generateAdjacencyMatrix(M2, vertices2);
printMatrix(M2, vertices2);

//Преобразование и вывод списка смежности для G2
matrixToList(M2, vertices2);

//Освобождение памяти
freeMatrix(M1, vertices1);
freeMatrix(M2, vertices2);
return 0;
}

```

Файл 6LR_1.cpp (2 задание 1)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

//Функция для генерации матрицы смежности
void generateAdjacencyMatrix(int** matrix, int vertices) {
    //Проходим по всем вершинам матрицы
    for (int i = 0; i < vertices; i++) {
        for (int j = i + 1; j < vertices; j++) {
            //Генерируем случайное значение 0 или 1 для ребра
            int value = rand() % 2;
            //Заполняем симметрично
            matrix[i][j] = value;
            matrix[j][i] = value;
        }
        matrix[i][i] = 0;
    }
}

//Функция для вывода матрицы смежности
void printMatrix(int** matrix, int vertices) {
    //Выводим заголовок с размерами матрицы
    printf("Матрица смежности (%dx%d):\n", vertices, vertices);

    //Выводим номера столбцов
    printf("    ");
    for (int j = 0; j < vertices; j++) {
        printf("%2d ", j + 1); //Нумерация вершин с 1 для удобства
    }
    printf("\n");

    //Выводим саму матрицу
    for (int i = 0; i < vertices; i++) {

```

```

        printf("%2d ", i + 1); //Номер строки
        for (int j = 0; j < vertices; j++) {
            printf("%2d ", matrix[i][j]); //Значение элемента матрицы
        }
        printf("\n");
    }
    printf("\n");
}

//Функция для копирования матрицы
int** copyMatrix(int** source, int vertices) {
    //Выделяем память для новой матрицы
    int** copy = (int**)malloc(vertices * sizeof(int*));
    for (int i = 0; i < vertices; i++) {
        //Выделяем память для каждой строки
        copy[i] = (int*)malloc(vertices * sizeof(int));
        //Копируем значения из исходной матрицы
        for (int j = 0; j < vertices; j++) {
            copy[i][j] = source[i][j];
        }
    }
    return copy;
}

//Операция отождествления вершин
int** identifyVertices(int** matrix, int vertices, int v1, int v2,
int* newVertices) {
    //Проверка корректности введенных вершин
    if (v1 < 1 || v1 > vertices || v2 < 1 || v2 > vertices || v1 ==
v2) {
        printf("Ошибка: некорректные номера вершин!\n");
        *newVertices = vertices;
        return copyMatrix(matrix, vertices); //Возвращаем копию
исходной матрицы
    }

    //Преобразуем в индексы (от 0 до v1)
    int idx1 = v1 - 1;
    int idx2 = v2 - 1;
    //Оставляем вершину с меньшим индексом
    int keep_idx = (idx1 < idx2) ? idx1 : idx2;
    int remove_idx = (idx1 < idx2) ? idx2 : idx1;
    //Новая матрица будет на одну вершину меньше
    *newVertices = vertices - 1;
    //Выделяем память для новой матрицы и инициализируем нулями
    int** newMatrix = (int**)malloc(*newVertices * sizeof(int*));
    for (int i = 0; i < *newVertices; i++) {
        newMatrix[i] = (int*)calloc(*newVertices, sizeof(int));
    }
    //Заполняем новую матрицу
    for (int i = 0; i < vertices; i++) {
        if (i == remove_idx) continue; //Пропускаем удаляемую вершину
        //Вычисляем новый индекс для текущей вершины
        int new_i = (i < remove_idx) ? i : i - 1;
        for (int j = 0; j < vertices; j++) {

```

```

        if (j == remove_idx) continue; //Пропускаем удаляемую
вершину
        //Вычисляем новый индекс для текущего столбца
        int new_j = (j < remove_idx) ? j : j - 1;
        if (i == keep_idx || j == keep_idx) {
            //Для оставляемой вершины объединяем связи
            if (i == keep_idx && j == keep_idx) {
                newMatrix[new_i][new_j] = 0; // Петли нет
            }
            else if (i == keep_idx) {
                //Объединяем связи из обеих вершин (логическое
ИЛИ)
                newMatrix[new_i][new_j] = (matrix[keep_idx][j]
|| matrix[remove_idx][j]) ? 1 : 0;
            }
            else if (j == keep_idx) {
                //Объединяем связи в обе вершины (логическое
ИЛИ)
                newMatrix[new_i][new_j] = (matrix[i][keep_idx]
|| matrix[i][remove_idx]) ? 1 : 0;
            }
        }
        else {
            //Просто копируем значение для остальных вершин
            newMatrix[new_i][new_j] = matrix[i][j];
        }
    }
}
return newMatrix;
}

//Операция стягивания ребра
int** contractEdge(int** matrix, int vertices, int v1, int v2, int*
newVertices) {
    //Проверка корректности введенных вершин
    if (v1 < 1 || v1 > vertices || v2 < 1 || v2 > vertices || v1 ==
v2) {
        printf("Ошибка: некорректные номера вершин!\n");
        *newVertices = vertices;
        return copyMatrix(matrix, vertices);
    }
    int idx1 = v1 - 1;
    int idx2 = v2 - 1;
    //Проверяем, существует ли ребро между вершинами
    if (matrix[idx1][idx2] == 0) {
        printf("Ошибка: ребро между вершинами %d и %d не
существует!\n", v1, v2);
        *newVertices = vertices;
        return copyMatrix(matrix, vertices);
    }
    //Стягивание ребра аналогично отождествлению вершин
    return identifyVertices(matrix, vertices, v1, v2, newVertices);
}

//Операция расщепления вершины

```

```

int** splitVertex(int** matrix, int vertices, int v, int*
newVertices) {
    //Проверка корректности введенной вершины
    if (v < 1 || v > vertices) {
        printf("Ошибка: некорректный номер вершины!\n");
        *newVertices = vertices;
        return copyMatrix(matrix, vertices);
    }
    int idx = v - 1;
    *newVertices = vertices + 1;
    //Создаем новую матрицу
    int** newMatrix = (int**)malloc(*newVertices * sizeof(int*));
    for (int i = 0; i < *newVertices; i++) {
        newMatrix[i] = (int*)calloc(*newVertices, sizeof(int));
    }
    //Копируем исходную матрицу без изменений
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            newMatrix[i][j] = matrix[i][j];
        }
    }
    int newVertexIdx = vertices; // Новая вершина имеет индекс
vertices (последняя)
    //Соединяем исходную и новую вершину (ребро между ними)
    newMatrix[idx][newVertexIdx] = 1;
    newMatrix[newVertexIdx][idx] = 1;
    //На диагонали нет петель
    for (int i = 0; i < *newVertices; i++) {
        newMatrix[i][i] = 0;
    }
    return newMatrix;
}

//Функция для освобождения памяти
void freeMatrix(int** matrix, int vertices) {
    // Освобождаем память для каждой строки
    for (int i = 0; i < vertices; i++) {
        free(matrix[i]);
    }
    //Освобождаем память для массива указателей
    free(matrix);
}

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(NULL));
    int vertices;
    printf("Введите количество вершин для графа: ");
    scanf("%d", &vertices);

    //Проверка корректности ввода
    if (vertices <= 0) {
        printf("Ошибка: количество вершин должно быть положительным
числом.\n");
        return 1;
    }
}

```

```

//Выделение памяти и генерация исходной матрицы
int** matrix = (int**)malloc(vertices * sizeof(int*));
for (int i = 0; i < vertices; i++) {
    matrix[i] = (int*)malloc(vertices * sizeof(int));
}

//Генерируем и выводим исходную матрицу
generateAdjacencyMatrix(matrix, vertices);
printf("\nИсходный граф\n");
printMatrix(matrix, vertices);
int choice;
int v1, v2, v;
int newVertices;
do {
    printf("\nОперации над графом \n");
    printf("1 - Отождествление вершин\n");
    printf("2 - Стягивание ребра\n");
    printf("3 - Расщепление вершины\n");
    printf("0 - Выход\n");
    printf("Выберите операцию: ");
    scanf("%d", &choice);
    //Обработка выбора пользователя
    switch (choice) {
        case 1: {
            //Операция отождествления вершин
            printf("Введите номера вершин для отождествления (через
пробел): ");
            scanf("%d %d", &v1, &v2);
            int** result = identifyVertices(matrix, vertices, v1,
v2, &newVertices);
            printf("\nРезультат отождествления вершин %d и %d:\n",
v1, v2);
            printMatrix(result, newVertices);
            freeMatrix(result, newVertices);
            break;
        }
        case 2: {
            //Операция стягивания ребра
            printf("Введите номера вершин ребра для стягивания
(через пробел): ");
            scanf("%d %d", &v1, &v2);
            int** result = contractEdge(matrix, vertices, v1, v2,
&newVertices);
            printf("\nРезультат стягивания ребра (%d, %d):\n", v1,
v2);
            printMatrix(result, newVertices);
            freeMatrix(result, newVertices);
            break;
        }
        case 3: {
            // Операция расщепления вершины
            printf("Введите номер вершины для расщепления: ");
            scanf("%d", &v);
            int** result = splitVertex(matrix, vertices, v,
&newVertices);

```

```

        printf("\nРезультат расщепления вершины %d:\n", v);
        printMatrix(result, newVertices);
        freeMatrix(result, newVertices);
        break;
    }
    case 0:
        printf("Выход из программы.\n");
        break;
    default:
        printf("Неверный выбор!\n");
    }
} while (choice != 0);
//Освобождение памяти исходной матрицы
freeMatrix(matrix, vertices);
return 0;
}

```

Файл Lr6_2_2.cpp (2 задание 2)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

//Структура для узла списка смежности
typedef struct Node {
    int vertex; // Номер смежной вершины
    struct Node* next; // Указатель на следующий узел
} Node;

//Структура для списка смежности графа
typedef struct {
    int vertices; // Количество вершин
    Node** adjLists; // Массив списков смежности
} Graph;

//Функция для создания нового узла
Node* createNode(int v) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

// Функция для создания графа
Graph* createGraph(int vertices) {
    Graph* graph = (Graph*)malloc(sizeof(Graph));
    graph->vertices = vertices;

    // Выделяем память для массива списков смежности
    graph->adjLists = (Node**)malloc(vertices * sizeof(Node*));

    // Инициализируем каждый список как пустой
    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
    }
}

```

```

    return graph;
}

//Функция для добавления ребра в неориентированный граф
void addEdge(Graph* graph, int src, int dest) {
    //Добавляем ребро из src в dest (исходная -> целевая)
    Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;
    //Добавляем ребро из dest в src (граф неориентированный)
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}

//Функция для генерации случайного графа
void generateRandomGraph(Graph* graph, int vertices) {
    //Генерируем случайные ребра
    for (int i = 0; i < vertices; i++) {
        for (int j = i + 1; j < vertices; j++) {
            if (rand() % 2 == 1) { // С вероятностью 50% добавляем
ребро
                addEdge(graph, i, j);
            }
        }
    }
}

//Функция для вывода списков смежности
void printAdjacencyList(Graph* graph) {
    printf("Списки смежности:\n");
    for (int i = 0; i < graph->vertices; i++) {
        printf("Вершина %d: ", i + 1);
        Node* temp = graph->adjLists[i];
        if (temp == NULL) {
            printf("нет соседей");
        }
        else {
            while (temp) {
                printf("%d ", temp->vertex + 1); //нумерация с 1
                temp = temp->next;
            }
        }
        printf("\n");
    }
    printf("\n");
}

//Функция для копирования графа
Graph* copyGraph(Graph* graph) {
    Graph* copy = createGraph(graph->vertices);
    //Копируем каждый список смежности
    for (int i = 0; i < graph->vertices; i++) {
        Node* temp = graph->adjLists[i];

```

```

        Node** copyPtr = &(copy->adjLists[i]); // Указатель на
текущий конец списка

        while (temp) {
            *copyPtr = createNode(temp->vertex); // Создаем копию
узла
            copyPtr = &((*copyPtr)->next); // Перемещаем указатель на
следующий элемент
            temp = temp->next;
        }
    }

    return copy;
}

//Функция для удаления вершины из списка смежности
void removeVertexFromList(Node** head, int v) {
    Node* temp = *head;
    Node* prev = NULL;
    //Ищем вершину в списке
    while (temp != NULL && temp->vertex != v) {
        prev = temp;
        temp = temp->next;
    }
    //Если вершина найдена, удаляем её
    if (temp != NULL) {
        if (prev == NULL) {
            *head = temp->next; // Удаляем первый элемент
        }
        else {
            prev->next = temp->next; // Удаляем из середины или конца
        }
        free(temp);
    }
}

//Функция для проверки наличия вершины в списке
int isVertexInList(Node* head, int v) {
    Node* temp = head;
    while (temp != NULL) {
        if (temp->vertex == v) {
            return 1; // Вершина найдена
        }
        temp = temp->next;
    }
    return 0; // Вершина не найдена
}

//Функция для освобождения памяти графа
void freeGraph(Graph* graph) {
    for (int i = 0; i < graph->vertices; i++) {
        Node* temp = graph->adjLists[i];
        while (temp) {
            Node* toDelete = temp;
            temp = temp->next;
            free(toDelete); // Освобождаем каждый узел списка
        }
    }
}

```

```

    }
}
free(graph->adjLists); // Освобождаем массив указателей
free(graph); // Освобождаем структуру графа
}

// а) Операция отождествления вершин
Graph* identifyVerticesList(Graph* graph, int v1, int v2) {
    //Проверка корректности входных данных
    if (v1 < 1 || v1 > graph->vertices || v2 < 1 || v2 > graph->vertices || v1 == v2) {
        printf("Ошибка: некорректные номера вершин!\n");
        return copyGraph(graph); // Возвращаем копию исходного графа
    }
    //Преобразуем номера вершин в индексы массива (начинаются с 0)
    int idx1 = v1 - 1;
    int idx2 = v2 - 1;
    //Определяем какую вершину оставляем, какую удаляем
    int keep_idx = (idx1 < idx2) ? idx1 : idx2;
    int remove_idx = (idx1 < idx2) ? idx2 : idx1;
    //Создаем новый граф с уменьшенным количеством вершин
    Graph* newGraph = createGraph(graph->vertices - 1);
    //Сначала собираем всех соседей объединенной вершины
    Node* combinedNeighbors = NULL;
    //Добавляем соседей вершины keep_idx
    Node* temp = graph->adjLists[keep_idx];
    while (temp != NULL) {
        //Исключаем петли и ссылки на удаляемую вершину
        if (temp->vertex != keep_idx && temp->vertex != remove_idx)
        {
            Node* newNode = createNode(temp->vertex);
            newNode->next = combinedNeighbors;
            combinedNeighbors = newNode; // Добавляем в начало списка
        }
        temp = temp->next;
    }

    //Добавляем соседей вершины remove_idx
    temp = graph->adjLists[remove_idx];
    while (temp != NULL) {
        if (temp->vertex != keep_idx && temp->vertex != remove_idx)
        {
            //Проверяем, нет ли уже такой вершины в списке
            if (!isVertexInList(combinedNeighbors, temp->vertex)) {
                Node* newNode = createNode(temp->vertex);
                newNode->next = combinedNeighbors;
                combinedNeighbors = newNode;
            }
        }
        temp = temp->next;
    }

    //Заполняем новый граф
    for (int i = 0; i < graph->vertices; i++) {
        if (i == remove_idx) continue; // Пропускаем удаляемую
        вершину
    }
}

```

```

        int new_i = (i < remove_idx) ? i : i - 1; // Корректируем
индекс
        Node* currentList = NULL;
        if (i == keep_idx) {
            //Для объединенной вершины используем собранный список
соседей
            currentList = combinedNeighbors;
        }
        else {
            //Для остальных вершин копируем список, заменяя индексы
            temp = graph->adjLists[i];
            while (temp != NULL) {
                int old_j = temp->vertex;
                int new_j;
                //Заменяем ссылки на удаляемую вершину ссылками на
оставляемую
                if (old_j == remove_idx) {
                    new_j = keep_idx;
                }
                else {
                    new_j = (old_j < remove_idx) ? old_j : old_j -
1;
                }
                //Добавляем ребро если это не петля и ребро еще не
существует
                if (new_j != new_i && !isVertexInList(currentList,
new_j)) {
                    Node* newNode = createNode(new_j);
                    newNode->next = currentList;
                    currentList = newNode;
                }
                temp = temp->next;
            }
            newGraph->adjLists[new_i] = currentList;
        }
        return newGraph;
    }

// 6) Операция стягивания ребра
Graph* contractEdgeList(Graph* graph, int v1, int v2) {
    //Проверка корректности входных данных
    if (v1 < 1 || v1 > graph->vertices || v2 < 1 || v2 > graph-
>vertices || v1 == v2) {
        printf("Ошибка: некорректные номера вершин!\n");
        return copyGraph(graph);
    }
    int idx1 = v1 - 1;
    int idx2 = v2 - 1;
    //Проверяем существование ребра между вершинами
    if (!isVertexInList(graph->adjLists[idx1], idx2)) {
        printf("Ошибка: ребро между вершинами %d и %d не
существует!\n", v1, v2);
        return copyGraph(graph);
    }
    //Стягивание ребра - это отождествление вершин + удаление петли

```

```

Graph* result = identifyVerticesList(graph, v1, v2);
//Удаляем петлю из объединенной вершины (если она есть)
int keep_idx = (idx1 < idx2) ? idx1 : idx1 - 1; // Учитываем
смещение индексов
removeVertexFromList(&(result->adjLists[keep_idx]), keep_idx);
return result;
}

// в) Операция расщепления вершины
Graph* splitVertexList(Graph* graph, int v) {
    // Проверка корректности введенной вершины
    if (v < 1 || v > graph->vertices) {
        printf("Ошибка: некорректный номер вершины!\n");
        return copyGraph(graph);
    }
    int idx = v - 1; // Преобразуем номер вершины в индекс
    //Создаем новый граф с +1 вершиной
    Graph* newGraph = createGraph(graph->vertices + 1);
    int newVertexIdx = graph->vertices; // Индекс новой вершины
    (последний)
    //Копируем все исходные списки смежности без изменений
    for (int i = 0; i < graph->vertices; i++) {
        Node* temp = graph->adjLists[i];
        while (temp != NULL) {
            int old_j = temp->vertex;
            //Копируем ребро без изменений
            if (!isVertexInList(newGraph->adjLists[i], old_j) && i
!= old_j) {
                Node* newNode = createNode(old_j);
                newNode->next = newGraph->adjLists[i];
                newGraph->adjLists[i] = newNode;
            }
            temp = temp->next;
        }
    }

    //Соединяем исходную вершину и новую вершину (ребро между ними)
    if (!isVertexInList(newGraph->adjLists[idx], newVertexIdx)) {
        Node* newNode1 = createNode(newVertexIdx);
        newNode1->next = newGraph->adjLists[idx];
        newGraph->adjLists[idx] = newNode1;

        Node* newNode2 = createNode(idx);
        newNode2->next = newGraph->adjLists[newVertexIdx];
        newGraph->adjLists[newVertexIdx] = newNode2;
    }

    return newGraph;
}

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(NULL)); // Инициализация генератора случайных чисел
    int vertices;
    printf("Введите количество вершин для графа: ");
    scanf("%d", &vertices);

```

```

    if (vertices <= 0) {
        printf("Ошибка: количество вершин должно быть положительным
числом.\n");
        return 1;
    }

    //Создаем и генерируем граф
    Graph* graph = createGraph(vertices);
    generateRandomGraph(graph, vertices);
    printf("\nИсходный граф\n");
    printAdjacencyList(graph);
    int choice;
    int v1, v2, v;
    //Основной цикл меню
    do {
        printf("\nОперации над графом\n");
        printf("1 - Отождествление вершин\n");
        printf("2 - Стыгивание ребра\n");
        printf("3 - Расщепление вершины\n");
        printf("0 - Выход\n");
        printf("Выберите операцию: ");
        scanf("%d", &choice);

        Graph* result = NULL;

        switch (choice) {
            case 1: {
                printf("Введите номера вершин для отождествления (через
пробел): ");
                scanf("%d %d", &v1, &v2);
                result = identifyVerticesList(graph, v1, v2);
                printf("\nРезультат отождествления вершин %d и %d:\n",
v1, v2);
                printAdjacencyList(result);
                break;
            }
            case 2: {
                printf("Введите номера вершин ребра для стыгивания
(через пробел): ");
                scanf("%d %d", &v1, &v2);
                result = contractEdgeList(graph, v1, v2);
                printf("\nРезультат стыгивания ребра (%d, %d):\n", v1,
v2);
                printAdjacencyList(result);
                break;
            }
            case 3: {
                printf("Введите номер вершины для расщепления: ");
                scanf("%d", &v);
                result = splitVertexList(graph, v);
                printf("\nРезультат расщепления вершины %d:\n", v);
                printAdjacencyList(result);
                break;
            }
            case 0:
                printf("Выход из программы.\n");

```

```

        break;
    default:
        printf("Неверный выбор!\n");
    }

    if (result != NULL) {
        freeGraph(result); // Освобождаем память результата
операции
    }

    } while (choice != 0);

    //Освобождение памяти исходного графа
    freeGraph(graph);
    return 0;
}

```

Файл LR6_3.cpp (3 задание)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

//Функция для генерации матрицы смежности
void generateAdjacencyMatrix(int** matrix, int vertices) {
    //Проходим по всем вершинам графа
    for (int i = 0; i < vertices; i++) {
        //Начинаем с j = i+1 чтобы избежать дублирования (граф
неориентированный)
        for (int j = i + 1; j < vertices; j++) {
            //Генерируем случайное значение 0 или 1 для наличия
ребра
            int value = rand() % 2;
            //Заполняем матрицу симметрично
            matrix[i][j] = value;
            matrix[j][i] = value;
        }
        //Устанавливаем 0 на главной диагонали
        matrix[i][i] = 0;
    }
}

//Функция для вывода матрицы смежности на экран
void printMatrix(int** matrix, int vertices, const char* name) {
    //Выводим название матрицы и ее размеры
    printf("%s (%dx%d):\n", name, vertices, vertices);
    //Выводим заголовок с номерами столбцов
    printf("    ");
    for (int j = 0; j < vertices; j++) {
        printf("%2d ", j + 1); // +1 для нумерации вершин с 1
    }
    printf("\n");

    //Выводим саму матрицу построчно
    for (int i = 0; i < vertices; i++) {

```

```

        printf("%2d ", i + 1); // Номер строки (вершины)
        for (int j = 0; j < vertices; j++) {
            printf("%2d ", matrix[i][j]); // Значение элемента
матрицы
        }
        printf("\n");
    }
    printf("\n");
}

//Функция для создания матрицы заданного размера
int** createMatrix(int vertices) {
    //Выделяем память для массива указателей на строки матрицы
    int** matrix = (int**)malloc(vertices * sizeof(int*));
    //Для каждой строки выделяем память и инициализируем нулями
    for (int i = 0; i < vertices; i++) {
        matrix[i] = (int*)calloc(vertices, sizeof(int)); // calloc
инициализирует нулями
    }
    return matrix;
}

//Функция для освобождения памяти, занятой матрицей
void freeMatrix(int** matrix, int vertices) {
    //Освобождаем память каждой строки матрицы
    for (int i = 0; i < vertices; i++) {
        free(matrix[i]);
    }
    //Освобождаем память массива указателей
    free(matrix);
}

//Функция для создания глубокой копии матрицы
int** copyMatrix(int** source, int vertices) {
    //Создаем новую матрицу того же размера
    int** copy = createMatrix(vertices);
    //Копируем все элементы из исходной матрицы
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            copy[i][j] = source[i][j];
        }
    }
    return copy;
}

// а) Операция объединения графов (ИЛИ)
int** unionGraphs(int** matrix1, int** matrix2, int vertices) {
    //Создаем результирующую матрицу
    int** result = createMatrix(vertices);
    //Проходим по всем элементам матриц
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            result[i][j] = (matrix1[i][j] || matrix2[i][j]) ? 1 : 0;
        }
    }
}

```

```

    return result;
}

// б) Операция пересечения графов (И)
int** intersectionGraphs(int** matrix1, int** matrix2, int vertices)
{
    //Создаем результирующую матрицу
    int** result = createMatrix(vertices);
    //Проходим по всем элементам матриц
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            result[i][j] = (matrix1[i][j] && matrix2[i][j]) ? 1 : 0;
        }
    }
    return result;
}

// в) Сложение по модулю 2
int** ringSumGraphs(int** matrix1, int** matrix2, int vertices) {
    //Создаем результирующую матрицу
    int** result = createMatrix(vertices);
    //Проходим по всем элементам матриц
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            //Ребро существует, если оно есть в G1 или в G2, но не в
обоих
            //1 если значения разные, 0 если одинаковые
            result[i][j] = (matrix1[i][j] != matrix2[i][j]) ? 1 : 0;
        }
    }
    return result;
}

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(NULL));
    int vertices;
    printf("Введите количество вершин для графов G1 и G2: ");
    scanf("%d", &vertices);

    //Проверяем корректность введенного значения
    if (vertices <= 0) {
        printf("Ошибка: количество вершин должно быть положительным
числом.\n");
        return 1; // Возвращаем код ошибки
    }
    int** matrix1 = createMatrix(vertices);
    int** matrix2 = createMatrix(vertices);

    //Заполняем матрицы случайными значениями
    generateAdjacencyMatrix(matrix1, vertices);
    generateAdjacencyMatrix(matrix2, vertices);
    //Выводим исходные графы
    printf("\nИсходные графы\n");
    printMatrix(matrix1, vertices, "Матрица смежности G1");
    printMatrix(matrix2, vertices, "Матрица смежности G2");
}

```

```

int choice;// Переменная для хранения выбора пользователя
do {
    printf("\nОперации над графами\n");
    printf("1 - Объединение G1 U G2\n");
    printf("2 - Пересечение G1 n G2\n");
    printf("3 - Сложение по модулю (2) G1 (+) G2\n");
    printf("0 - Выход\n");
    printf("Выберите операцию: ");
    scanf("%d", &choice);

    //Указатель на результирующую матрицу
    int** result = NULL;
    //Название операции для вывода
    const char* operationName = "";
    switch (choice) {
        case 1: {
            result = unionGraphs(matrix1, matrix2, vertices);
            operationName = "Результат объединения G1 U G2";
            break;
        }
        case 2: {
            result = intersectionGraphs(matrix1, matrix2, vertices);
            operationName = "Результат пересечения G1 n G2";
            break;
        }
        case 3: {
            result = ringSumGraphs(matrix1, matrix2, vertices);
            operationName = "Результат сложения по модулю (2) G1 (+)
G2";
            break;
        }
        case 0:
            printf("Выход из программы.\n");
            break;
        default:
            printf("Неверный выбор!\n");
    }

    //Если операция была выполнена
    if (result != NULL) {
        //Выводим результат операции
        printf("\n%s\n", operationName);
        printMatrix(result, vertices, "");
        //Освобождаем память, занятую результатом
        freeMatrix(result, vertices);
    }
} while (choice != 0);
//Освобождение памяти исходных матриц
freeMatrix(matrix1, vertices);
freeMatrix(matrix2, vertices);
return 0;
}

```

Файл LR6_4.cpp (4 задание)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

```

```

#include <stdlib.h>
#include <time.h>
#include <locale.h>

//Функция для генерации матрицы смежности
void generateAdjacencyMatrix(int** matrix, int vertices) {
    for (int i = 0; i < vertices; i++) {
        for (int j = i + 1; j < vertices; j++) {
            //Генерируем случайное значение 0 или 1 для ребра между
            //вершинами i и j
            int value = rand() % 2;
            //Заполняем матрицу симметрично (граф неориентированный)
            matrix[i][j] = value;
            matrix[j][i] = value;
        }
        //Обнуляем главную диагональ
        matrix[i][i] = 0;
    }
}

//Функция для вывода матрицы смежности
//Выводит матрицу в удобочитаемом формате с заголовками строк и
//столбцов
void printMatrix(int** matrix, int vertices, const char* name) {
    printf("%s (%dx%d):\n", name, vertices, vertices);
    //Вывод заголовков столбцов (номера вершин, начиная с 1)
    printf("    ");
    for (int j = 0; j < vertices; j++) {
        printf("%2d ", j + 1);
    }
    printf("\n");

    //Вывод самой матрицы
    for (int i = 0; i < vertices; i++) {
        printf("%2d ", i + 1); //Номер строки (вершины)
        for (int j = 0; j < vertices; j++) {
            printf("%2d ", matrix[i][j]); //Значение элемента матрицы
        }
        printf("\n");
    }
    printf("\n");
}

// Функция для создания матрицы
int** createMatrix(int vertices) {
    //Выделяем память для массива указателей на строки
    int** matrix = (int**)malloc(vertices * sizeof(int*));
    for (int i = 0; i < vertices; i++) {
        //Выделяем память для каждой строки и инициализируем нулями
        matrix[i] = (int*)calloc(vertices, sizeof(int));
    }
    return matrix;
}

// Функция для освобождения памяти матрицы
void freeMatrix(int** matrix, int vertices) {

```

```

    for (int i = 0; i < vertices; i++) {
        free(matrix[i]); // Освобождаем каждую строку
    }
    free(matrix); // Освобождаем массив указателей
}

// Операция декартова произведения графов  $G = G_1 \times G_2$ 
// Каждая вершина - это пара вершин из исходных графов
int** cartesianProduct(int** matrix1, int vertices1, int** matrix2,
int vertices2, int* resultVertices) {
    // Количество вершин в результате:  $V_1 \times V_2$ 
    *resultVertices = vertices1 * vertices2;
    int** result = createMatrix(*resultVertices);

    // Проходим по всем парам вершин (u,v) где u из  $G_1$ , v из  $G_2$ 
    for (int u1 = 0; u1 < vertices1; u1++) {
        for (int v1 = 0; v1 < vertices2; v1++) {
            //  $u_2, v_2$  - вторая вершина в декартовом произведении
            for (int u2 = 0; u2 < vertices1; u2++) {
                for (int v2 = 0; v2 < vertices2; v2++) {
                    // Индексы вершин в результирующем графе
                    // Используем формулу: индекс =  $u * vertices_2 + v$ 
                    int i = u1 * vertices2 + v1;
                    int j = u2 * vertices2 + v2;
                    // Две вершины  $(u_1, v_1)$  и  $(u_2, v_2)$  смежны в
                    // декартовом произведении если:
                    // 1)  $u_1 = u_2$  и  $(v_1, v_2)$  - ребро в  $G_2$ 
                    // (горизонтальные связи), ИЛИ
                    // 2)  $v_1 = v_2$  и  $(u_1, u_2)$  - ребро в  $G_1$ 
                    // (вертикальные связи)
                    if ((u1 == u2 && matrix2[v1][v2] == 1) || (v1 ==
                    v2 && matrix1[u1][u2] == 1)) {
                        result[i][j] = 1;
                    }
                }
            }
        }
    }
    return result;
}

// Функция для вывода матрицы
void printCartesianMatrix(int** matrix, int vertices1, int
vertices2, const char* name) {
    int totalVertices = vertices1 * vertices2;
    printf("%s (%dx%d):\n", name, totalVertices, totalVertices);
    printf("    ");
    for (int u = 0; u < vertices1; u++) {
        for (int v = 0; v < vertices2; v++) {
            printf("(%d,%d) ", u + 1, v + 1);
        }
    }
    printf("\n");

    // Вывод матрицы построчно
    for (int u1 = 0; u1 < vertices1; u1++) {

```

```

    for (int v1 = 0; v1 < vertices2; v1++) {
        int i = u1 * vertices2 + v1;
        printf("(%d,%d)", u1 + 1, v1 + 1); //Метка строки
        //Вывод всех элементов строки
        for (int u2 = 0; u2 < vertices1; u2++) {
            for (int v2 = 0; v2 < vertices2; v2++) {
                int j = u2 * vertices2 + v2;
                printf("%2d    ", matrix[i][j]);
            }
        }
        printf("\n");
    }
    printf("\n");
}

//Функция для вывода списка смежности
void printCartesianAdjacencyList(int** matrix, int vertices1, int
vertices2) {
    int totalVertices = vertices1 * vertices2;
    printf("Списки смежности декартова произведения:\n");
    //Для каждой вершины в декартовом произведении
    for (int u1 = 0; u1 < vertices1; u1++) {
        for (int v1 = 0; v1 < vertices2; v1++) {
            int i = u1 * vertices2 + v1;
            printf("Вершина (%d,%d): ", u1 + 1, v1 + 1);
            int hasNeighbors = 0; //Флаг наличия соседей
            //Ищем всех соседей текущей вершины
            for (int u2 = 0; u2 < vertices1; u2++) {
                for (int v2 = 0; v2 < vertices2; v2++) {
                    int j = u2 * vertices2 + v2;
                    //Если есть ребро и это не та же самая вершина
                    if (matrix[i][j] == 1 && i != j) {
                        printf("(%d,%d) ", u2 + 1, v2 + 1);
                        hasNeighbors = 1;
                    }
                }
            }
            //Если соседей нет, выводим соответствующее сообщение
            if (!hasNeighbors) {
                printf("Нет соседей");
            }
            printf("\n");
        }
    }
    printf("\n");
}

int main() {
    setlocale(LC_ALL, "ru");
    srand(time(NULL));
    int vertices1, vertices2;
    printf("Введите количество вершин для графа G1: ");
    scanf("%d", &vertices1);
    printf("Введите количество вершин для графа G2: ");
    scanf("%d", &vertices2);

```

```

//Проверка корректности введенных данных
if (vertices1 <= 0 || vertices2 <= 0) {
    printf("Ошибка: количество вершин должно быть положительным
числом.\n");
    return 1;
}
//Создаем и генерируем матрицы для G1 и G2
int** matrix1 = createMatrix(vertices1);
int** matrix2 = createMatrix(vertices2);

//Заполняем матрицы случайными значениями
generateAdjacencyMatrix(matrix1, vertices1);
generateAdjacencyMatrix(matrix2, vertices2);

//Вывод исходных графов
printf("\nИсходные графы\n");
printMatrix(matrix1, vertices1, "Матрица смежности G1");
printMatrix(matrix2, vertices2, "Матрица смежности G2");

//Выполняем декартово произведение
int resultVertices;
int** result = cartesianProduct(matrix1, vertices1, matrix2,
vertices2, &resultVertices);

//Вывод информации о результате
printf("\nДекартово произведение G1 X G2\n");
printf("Количество вершин в результате: %d x %d = %d\n\n",
vertices1, vertices2, resultVertices);

//Выводим матрицу смежности результата
printCartesianMatrix(result, vertices1, vertices2, "Матрица
смежности декартова произведения");

//Выводим списки смежности для наглядности
printCartesianAdjacencyList(result, vertices1, vertices2);

//Освобождение памяти
freeMatrix(matrix1, vertices1);
freeMatrix(matrix2, vertices2);
freeMatrix(result, resultVertices);
return 0;
}

```

Текст к программе

1. 6LR_1.cpp (1 задание)

Данная программа реализует генерацию и преобразование представлений неориентированных графов без петель. Метод решения включает следующие этапы: для начала программа запрашивает у пользователя количество вершин для двух графов G1 и G2, затем выделяет память под матрицы смежности для каждого графа. Матрицы заполняются случайными значениями с использованием генератора псевдослучайных чисел, при этом обеспечивается симметричность матриц (так как графы неориентированные) и отсутствие путей на главной диагонали. После генерации матриц программа выводит их

на экран в виде таблиц, где строки и столбцы соответствуют вершинам графа. Далее выполняется преобразование матричного представления в списки смежности: для каждой вершины формируется список смежных вершин путем анализа соответствующей строки матрицы, при этом проверяется наличие ребер (значения 1) и исключаются петли. Результаты преобразования выводятся в виде списков, где для каждой вершины перечислены все ее соседи или указывается отсутствие соседей. В завершение работы программа освобождает всю выделенную динамическую память.

2. Lr6_2_1_star.cpp (2 задание 1)

Данная программа реализует операции над неориентированными графами без петель в матричном представлении. Метод решения включает следующие этапы: программа запрашивает у пользователя количество вершин графа, затем выделяет память под матрицу смежности и заполняет ее случайными значениями с обеспечением симметричности. После генерации исходная матрица выводится на экран. Далее предоставляется меню операций: отождествление вершин, стягивание ребра и расщепление вершины. При отождествлении создается новая матрица уменьшенного размера, где связи объединяемых вершин комбинируются через логическое ИЛИ. При стягивании ребра выполняется проверка существования ребра и применяется модифицированное отождествление. При расщеплении создается матрица увеличенного размера со случайным распределением ребер между исходной и новой вершиной. Результаты каждой операции выводятся в виде матрицы смежности. В завершение программа освобождает всю выделенную динамическую память.

3. Lr6_2_2.cpp (2 задание 2)

Данная программа реализует операции над неориентированными графами в представлении списков смежности. Метод решения включает следующие этапы: программа запрашивает у пользователя количество вершин графа, затем создает структуру графа с массивом связанных списков для хранения смежности вершин. Граф заполняется случайными ребрами с вероятностью 50% для каждой возможной пары вершин. После генерации исходный граф выводится в виде списков смежности. Далее предоставляется меню операций над графом. При отождествлении вершин создается новый граф с уменьшенным количеством вершин, где формируется объединенный список соседей для сохраняемой вершины, а в остальных списках заменяются ссылки на удаляемую вершину. При стягивании ребра выполняется проверка существования ребра и применяется модифицированное отождествление с удалением возможной петли. При расщеплении вершины создается граф с увеличенным количеством вершин со случайным распределением ребер между исходной и новой вершиной и обязательным добавлением соединяющего ребра. Все операции сохраняют неориентированность графа через симметричное добавление ребер. Результаты каждой операции выводятся в виде списков смежности. В завершение программа освобождает всю

выделенную динамическую память, включая узлы связанных списков.

4. LR6_3.cpp (3 задание)

Данная программа реализует операции над неориентированными графами в матричном представлении. Метод решения включает следующие этапы: программа запрашивает у пользователя количество вершин для двух графов G1 и G2, затем выделяет память под матрицы смежности для каждого графа. Матрицы заполняются случайными значениями с использованием генератора псевдослучайных чисел, при этом обеспечивается симметричность матриц (так как графы неориентированные). После генерации матриц программа выводит их на экран в виде таблиц, где строки и столбцы соответствуют вершинам графа. Далее предоставляется меню операций над графами: объединение, пересечение и сложение по модулю 2. При объединении графов создается результирующая матрица, где каждый элемент получается применением логической операции ИЛИ к соответствующим элементам исходных матриц. При пересечении графов используется логическая операция И для соответствующих элементов матриц. При сложении по модулю 2 ребро сохраняется только если оно присутствует ровно в одном из графов. Все операции сохраняют свойство неориентированности графов через симметричное заполнение матриц. Результаты каждой операции выводятся в виде матрицы смежности. В завершение работы программа освобождает всю выделенную динамическую память.

5. LR6_4.cpp (4 задание)

Данная программа реализует операцию декартова произведения для неориентированных графов в матричном представлении. Метод решения включает следующие этапы: программа запрашивает у пользователя количество вершин для двух графов G1 и G2, затем выделяет память под матрицы смежности для каждого графа. Матрицы заполняются случайными значениями с использованием генератора псевдослучайных чисел, при этом обеспечивается симметричность матриц и отсутствие петель на главной диагонали. После генерации матриц программа выводит их на экран в виде таблиц. Далее выполняется операция декартова произведения: создается результирующая матрица размером $|V1| \times |V2|$, где каждая вершина представляет собой пару вершин из исходных графов. Алгоритм перебирает все возможные комбинации вершин $(u1, v1)$ и $(u2, v2)$ из G1 и G2, вычисляет соответствующие индексы в результирующей матрице по формуле $i = u1 \times |v2| + v1$, и устанавливает связь между вершинами при выполнении условий смежности - если $u1 = u2$ и существует ребро $(v1, v2)$ в G2 (горизонтальные связи), либо если $v1 = v2$ и существует ребро $(u1, u2)$ в G1 (вертикальные связи). Результат операции выводится в двух формах: в виде матрицы смежности с обозначением вершин в формате (u, v) и в виде списков смежности для наглядности. В завершение работы программа освобождает всю выделенную динамическую память.

Результаты работы программы

Результаты работы программы показаны на рисунках 2-3.

I. 6LR_1.cpp (1 задание)

```

Введите количество вершин для графа G1: 5
Введите количество вершин для графа G2: 5

Граф G1
Матрица смежности:
0 0 0 0 1
0 0 0 0 1
0 0 0 1 0
0 0 1 0 1
1 1 0 1 0

Список смежности:
Вершина 1: 5
Вершина 2: 5
Вершина 3: 4
Вершина 4: 3 5
Вершина 5: 1 2 4

Граф G2
Матрица смежности:
0 1 1 0 0
1 0 1 1 0
1 1 0 0 0
0 1 0 0 1
0 0 0 1 0

Список смежности:
Вершина 1: 2 3
Вершина 2: 1 3 4
Вершина 3: 1 2
Вершина 4: 2 5
Вершина 5: 4

```

Рисунок 1 – Корректная работа программы

```

Введите количество вершин для графа G1: 0
Введите количество вершин для графа G2: П
Ошибка: количество вершин должно быть вещественным числом.

```

Рисунок 2 – Попытка ввести 0 и букву в поля, отвечающие за кол-во вершин

```

Введите количество вершин для графа G1: -28
Введите количество вершин для графа G2: -11
Ошибка: количество вершин должно быть вещественным числом.

```

Рисунок 3 – Попытка ввести 0 и букву в поля, отвечающие за кол-во вершин

II. Lr6_2_1.cpp (2 задание 1)

```

Введите количество вершин для графа: 7

Исходный граф
Матрица смежности (7x7):
      1  2  3  4  5  6  7
1  0  0  1  1  1  0  1
2  0  0  1  1  0  0  1
3  1  1  0  0  1  1  0
4  1  1  0  0  1  0  1
5  1  0  1  1  0  1  1
6  0  0  1  0  1  0  1
7  1  1  0  1  1  1  0

Операции над графом
1 – Отождествление вершин
2 – Стягивание ребра
3 – Расщепление вершины
0 – Выход
Выберите операцию:

```

Рисунок 4 – Создание матрицы 7 на 7

```

Операции над графом
1 – Отождествление вершин
2 – Стягивание ребра
3 – Расщепление вершины
0 – Выход
Выберите операцию: 1
Введите номера вершин для отождествления (через пробел): 1
4

Результат отождествления вершин 1 и 4:
Матрица смежности (6x6):
      1  2  3  4  5  6
1  0  1  1  1  0  1
2  1  0  1  0  0  1
3  1  1  0  1  1  0
4  1  0  1  0  1  1
5  0  0  1  1  0  1
6  1  1  0  1  1  0

```

Рисунок 5 – Отождествление вершин 1 и 4

```

Операции над графом
1 – Отождествление вершин
2 – Стягивание ребра
3 – Расщепление вершины
0 – Выход
Выберите операцию: 2
Введите номера вершин ребра для стягивания (через пробел): 5
7

Результат стягивания ребра (5, 7):
Матрица смежности (6x6):
      1  2  3  4  5  6
1  0  0  1  1  1  0
2  0  0  1  1  1  0
3  1  1  0  0  1  1
4  1  1  0  0  1  0
5  1  1  1  1  0  1
6  0  0  1  0  1  0

```

Рисунок 6 – Стягивание ребра 5-7

```

Операции над графом
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Выход
Выберите операцию: 3
Введите номер вершины для расщепления: 4

Результат расщепления вершины 4:
Матрица смежности (8x8):
  1  2  3  4  5  6  7  8
1  0  0  1  1  0  1  0  1
2  0  0  1  0  0  0  0  1
3  1  1  0  0  0  1  1  0
4  1  0  0  0  0  0  0  1
5  0  0  0  0  0  0  0  0
6  1  0  1  0  0  0  1  1
7  0  0  1  0  0  1  0  1
8  1  1  0  1  0  1  1  0

```

Рисунок 7 – Расщепление 4 вершины

III. Lr6_2_2.cpp (2 задание 2)

```

Введите количество вершин для графа: 9

Исходный граф
Списки смежности:
Вершина 1: 9 8 4 3
Вершина 2: 8 7 5 3
Вершина 3: 9 6 4 2 1
Вершина 4: 9 3 1
Вершина 5: 8 6 2
Вершина 6: 5 3
Вершина 7: 9 8 2
Вершина 8: 9 7 5 2 1
Вершина 9: 8 7 4 3 1

```

Рисунок 8 – Создание матрицы 9 на 9

```

Операции над графом
1 - Отождествление вершин
2 - Стягивание ребра
3 - Расщепление вершины
0 - Выход
Выберите операцию: 1
Введите номера вершин для отождествления (через пробел): 4 9

Результат отождествления вершин 4 и 9:
Списки смежности:
Вершина 1: 3 8 4
Вершина 2: 3 5 7 8
Вершина 3: 1 2 6 4
Вершина 4: 7 8 1 3
Вершина 5: 2 6 8
Вершина 6: 3 5
Вершина 7: 2 8 4
Вершина 8: 1 2 5 7 4

```

Рисунок 9 – Отождествление вершин 1 и 4

```

Операции над графом
1 - Отождествление вершин
2 - Стыгивание ребра
3 - Расщепление вершины
0 - Выход
Выберите операцию: 2
Введите номера вершин ребра для стыгивания (через пробел): 1
8

Результат стыгивания ребра (1, 8):
Списки смежности:
Вершина 1: 2 5 7 3 4 9
Вершина 2: 3 5 7 1
Вершина 3: 1 2 4 6 8
Вершина 4: 1 3 8
Вершина 5: 2 6 1
Вершина 6: 3 5
Вершина 7: 2 1 8
Вершина 8: 3 4 7 1

```

Рисунок 10 – Стыгивание ребра 1 и 8

```

Операции над графом
1 - Отождествление вершин
2 - Стыгивание ребра
3 - Расщепление вершины
0 - Выход
Выберите операцию: 3
Введите номер вершины для расщепления: 9

Результат расщепления вершины 9:
Списки смежности:
Вершина 1: 3 4 8 9
Вершина 2: 3 5 7 8
Вершина 3: 1 2 4 6 10
Вершина 4: 1 3 10
Вершина 5: 2 6 8
Вершина 6: 3 5
Вершина 7: 2 8 9
Вершина 8: 1 2 5 7 9
Вершина 9: 10 7 8
Вершина 10: 9 1 3 4

```

Рисунок 11 – Расщепление 9 вершины

```

Операции над графом
1 - Отождествление вершин
2 - Стыгивание ребра
3 - Расщепление вершины
0 - Выход
Выберите операцию: 2
Введите номера вершин ребра для стыгивания (через пробел): 1
2
Ошибка: ребро между вершинами 1 и 2 не существует!

```

Рисунок 12 – Попытка стянуть вершины без ребра

IV. LR6_3.cpp (3 задание)

```

Введите количество вершин для графов G1 и G2: 5

Исходные графы
Матрица смежности G1 (5x5):
  1  2  3  4  5
1  0  0  1  0  1
2  0  0  1  1  0
3  1  1  0  1  1
4  0  1  1  0  1
5  1  0  1  1  0

Матрица смежности G2 (5x5):
  1  2  3  4  5
1  0  1  0  0  1
2  1  0  0  1  0
3  0  0  0  0  0
4  0  1  0  0  0
5  1  0  0  0  0

```

Рисунок 13 – Создание 2 графов размерами 5 на 5

```

Операции над графами
1 – Объединение G1 U G2
2 – Пересечение G1 n G2
3 – Сложение по модулю (2) G1 (+) G2
0 – Выход
Выберите операцию: 1

Результат объединения G1 U G2
(5x5):
  1  2  3  4  5
1  0  1  1  0  1
2  1  0  1  1  0
3  1  1  0  1  1
4  0  1  1  0  1
5  1  0  1  1  0

```

Рисунок 14 – Объединение матриц (ИЛИ, +)

```

Операции над графами
1 – Объединение G1 U G2
2 – Пересечение G1 n G2
3 – Сложение по модулю (2) G1 (+) G2
0 – Выход
Выберите операцию: 2

Результат пересечения G1 n G2
(5x5):
  1  2  3  4  5
1  0  0  0  0  1
2  0  0  0  1  0
3  0  0  0  0  0
4  0  1  0  0  0
5  1  0  0  0  0

```

Рисунок 15 – Объединение матриц (И, *)

```

Операции над графами
1 – Объединение G1 U G2
2 – Пересечение G1 n G2
3 – Сложение по модулю (2) G1 (+) G2
0 – Выход
Выберите операцию: 3

Результат сложения по модулю (2) G1 (+) G2
(5x5):
  1  2  3  4  5
1  0  1  1  0  0
2  1  0  1  0  0
3  1  1  0  1  1
4  0  0  1  0  1
5  0  0  1  1  0

```

Рисунок 16 – Сложение по модулю 2

V. LR6_4.cpp (4 задание)

```

Введите количество вершин для графа G1: 2
Введите количество вершин для графа G2: 3

Исходные графы
Матрица смежности G1 (2x2):
  1  2
1  0  1
2  1  0

Матрица смежности G2 (3x3):
  1  2  3
1  0  0  1
2  0  0  1
3  1  1  0

Декартово произведение G1 X G2
Количество вершин в результате: 2 x 3 = 6

Матрица смежности декартова произведения (6x6):
  (1,1) (1,2) (1,3) (2,1) (2,2) (2,3)
(1,1) 0      0      1      1      0      0
(1,2) 0      0      1      0      1      0
(1,3) 1      1      0      0      0      1
(2,1) 1      0      0      0      0      1
(2,2) 0      1      0      0      0      1
(2,3) 0      0      1      1      1      0

Списки смежности декартова произведения:
Вершина (1,1): (1,3) (2,1)
Вершина (1,2): (1,3) (2,2)
Вершина (1,3): (1,1) (1,2) (2,3)
Вершина (2,1): (1,1) (2,3)
Вершина (2,2): (1,2) (2,3)
Вершина (2,3): (1,3) (2,1) (2,2)

```

Рисунок 17 – Корректная работа программы

Выводы

В ходе выполнения лабораторной работы были успешно реализованы и протестированы основные операции над графами, представленными в двух формах: матрица смежности и списки смежности. Разработанная программа позволяет генерировать случайные графы, выполнять над ними унарные операции (отождествление вершин, стягивание ребра, расщепление вершины), а также бинарные операции (объединение, пересечение, кольцевая сумма и декартово произведение). Особенностью реализации является поддержка петель в графах и обеспечение корректности выполняемых операций. Интерфейс программы предоставляет удобное меню для выбора операций и демонстрации результатов в наглядной форме. В процессе работы подтверждена эквивалентность результатов выполнения одних и тех же операций над разными представлениями графов, что свидетельствует о корректности реализованных алгоритмов.